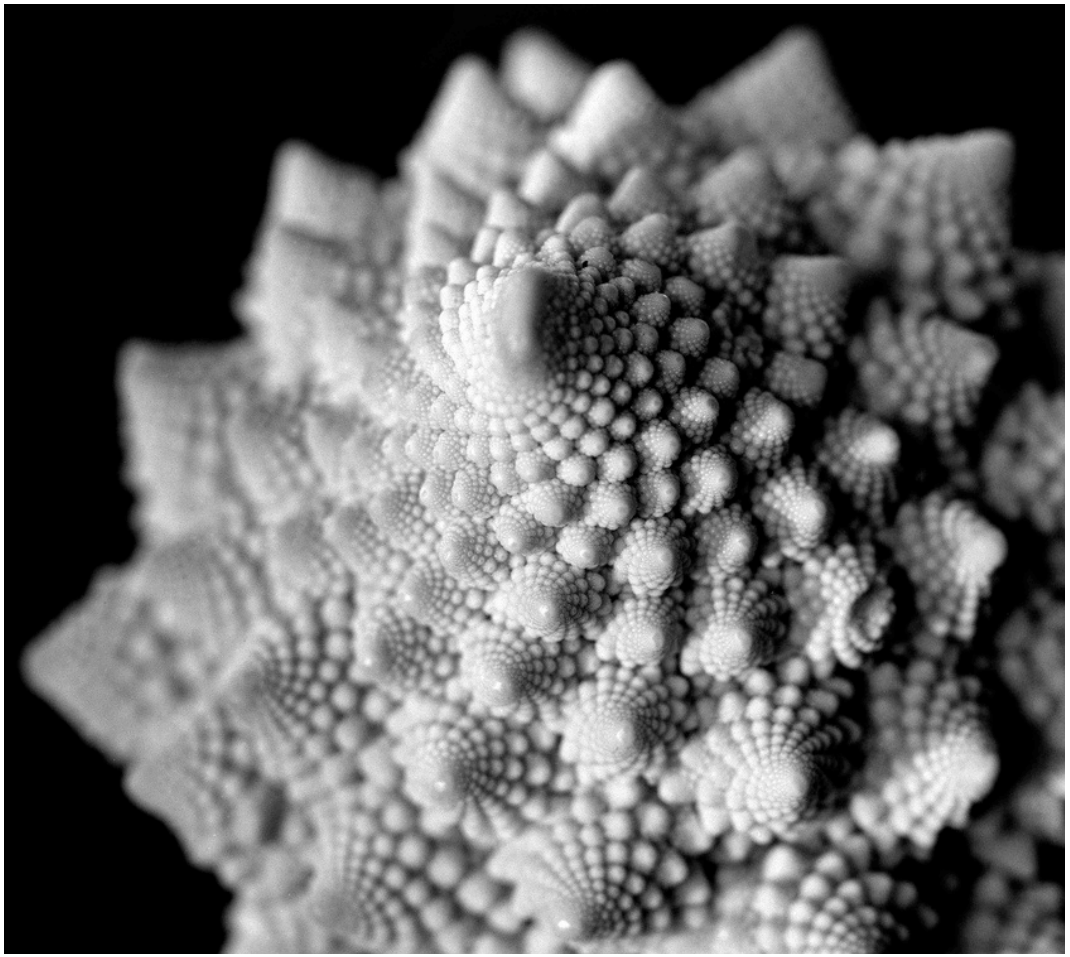
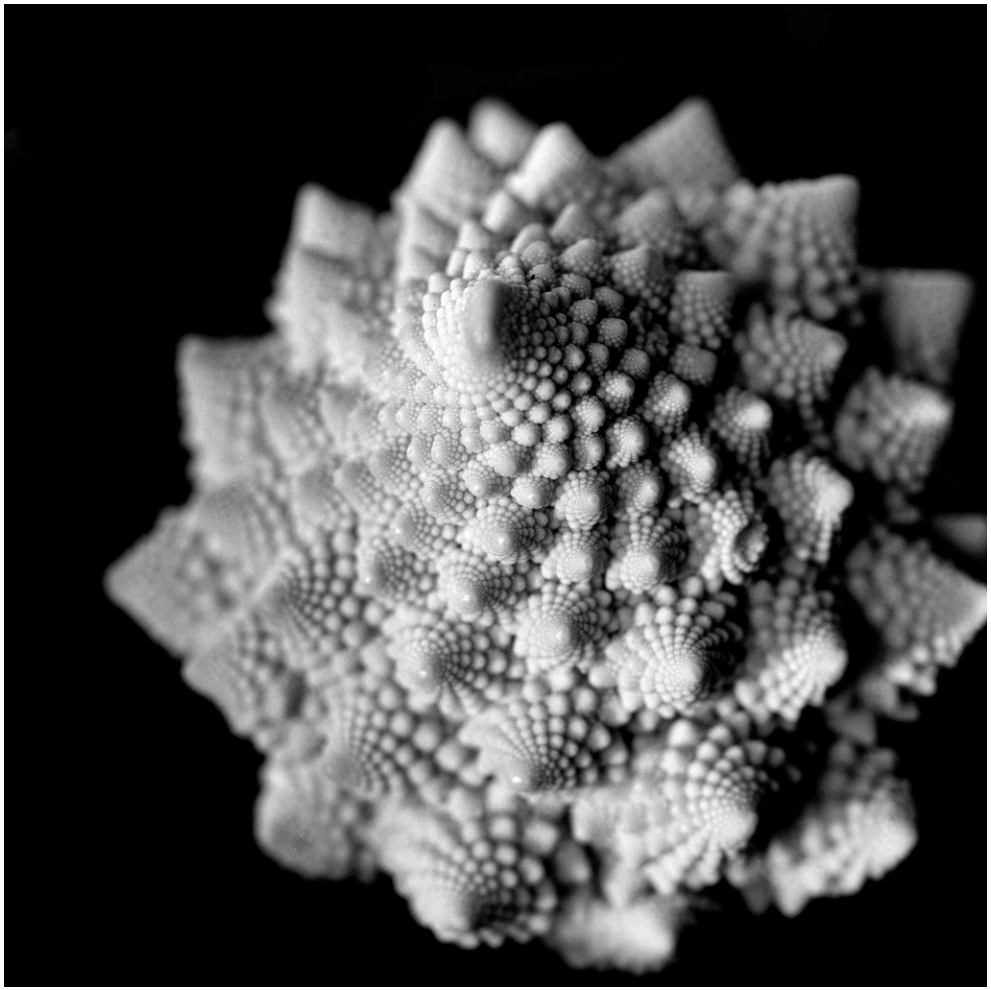
Creating iOS Table Views with Storyboards



TIM DUCKETT

Creating iOS Table Views with Storyboards



© 2012 Tim Duckett. Some Rights Reserved.

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License.

Introduction

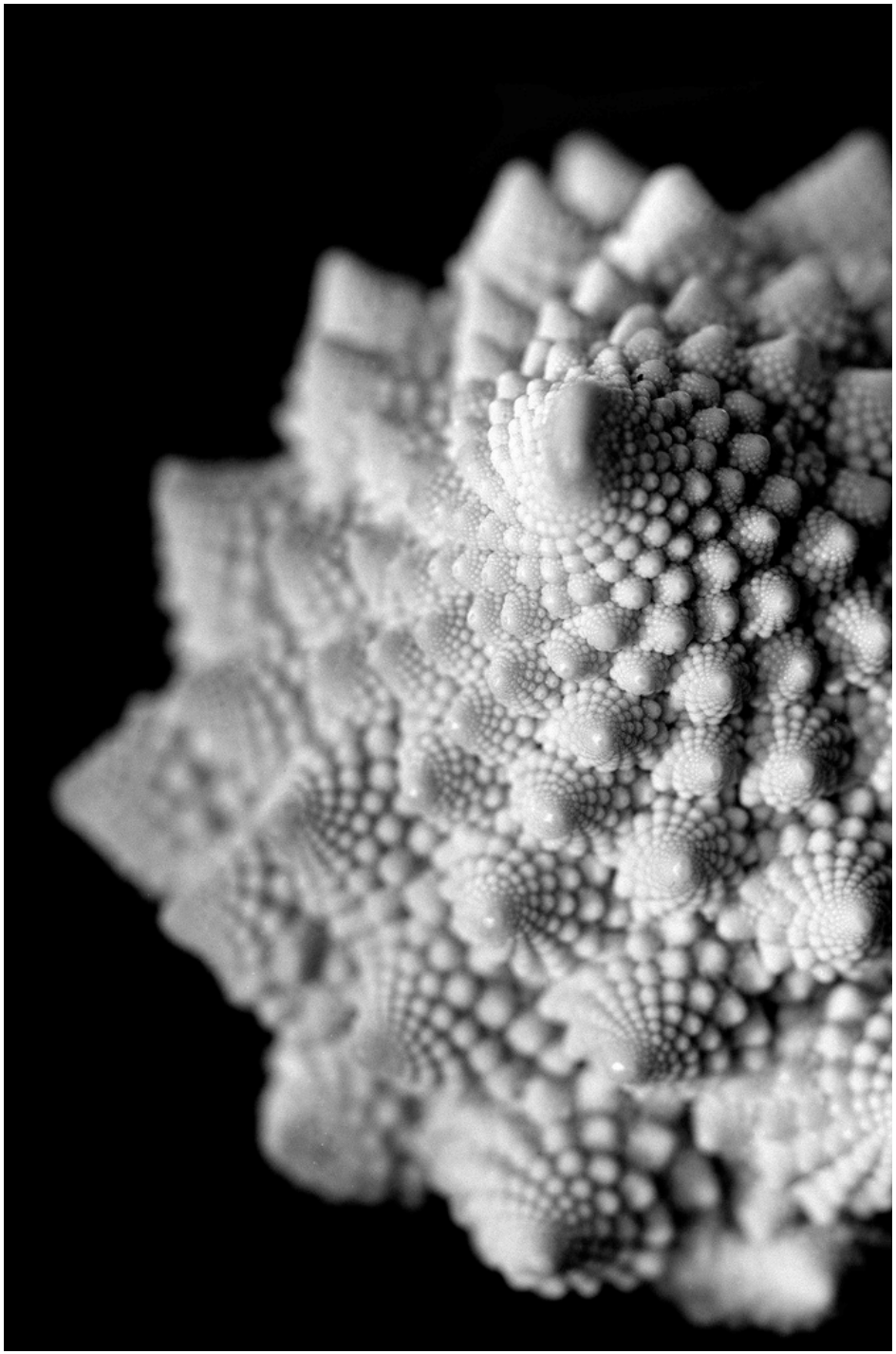
This guide is a quick-start introduction to using iOS5's Storyboard feature with table views.

It covers:

- What Storyboards are
- The basics of using Storyboards
- Using tableViews with Storyboards
- Customizing tableView cells with Storyboards

It follows on from the topics covered in my book "*Pro iOS Table Views for iPhone, iPad and iPod Touch*", published by Apress.

Code examples are available on GitHub at https://github.com/timd/Pro-iOS-TableViews



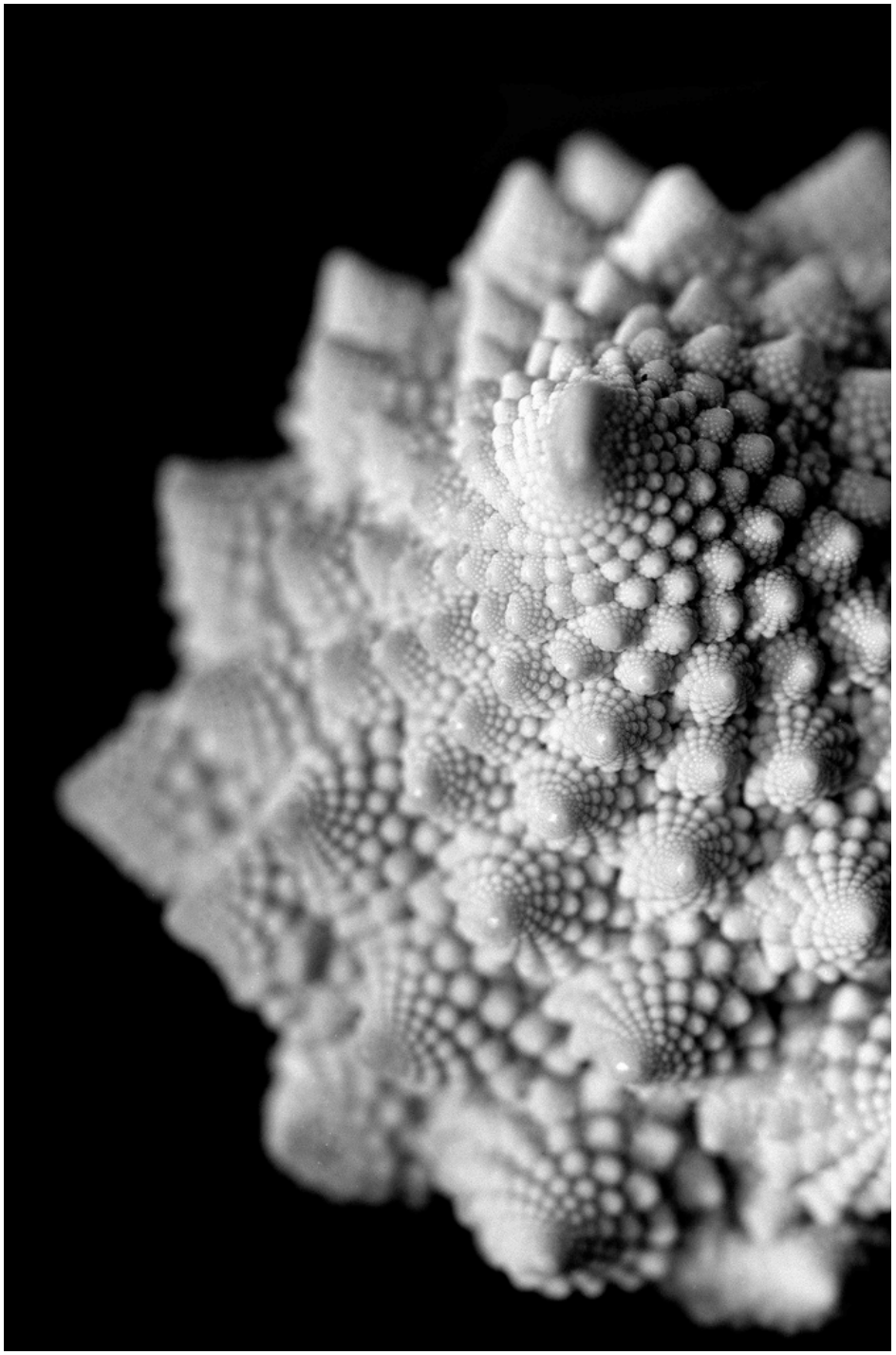
Chapter 2

Table Views and Storyboards

Storyboards are a new feature introduced in iOS5 and Xcode 4.2 that take the visual creation of interfaces to the next stage.

Although Interface Builder provides a visual way of laying out user interfaces, up until now the process of moving between screens has required a certain amount of “glue code”.

This is relatively straight-forward when you’re dealing with simple apps – but add UIKit functionality like tab bar controllers into the mix, and things suddenly get a lot trickier. Storyboards will come to your rescue in situations like this!



1

Using Table Views with Storyboards

Storyboards come in two parts – scenes, which are the equivalent of NIB files if you were building a non-Storyboard, and seques, which control the transition between scenes. This is conceptually similar to presentation apps like Keynote – scenes are like slides, and seques are like the transitions between the slides

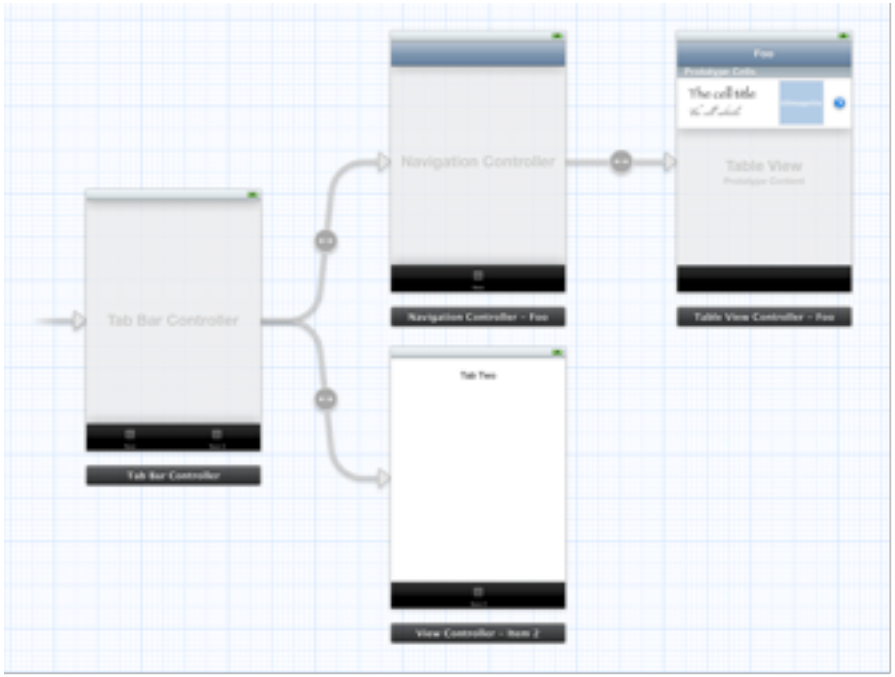


Figure 1 – Laying out a user journey with storyboards

To illustrate the process, let's create a simple application that brings all the concepts together. We'll build an app with a tab bar that has two views – one is a simple vanilla view controller, and the other is a tableView. For the moment, we'll keep it straight-forward and use generic cells – we'll customize these later on.

There are going to be 4 stages:

1. Creating a new app that uses storyboards
2. Adding a tab bar controller to hold the table view
3. Adding the navigation controller and table view controller
4. Displaying data in the table view

Creating the new app

We're going to need a Single View application that uses Storyboards. From the File menu, select File > New > Project, and select the Single View Application template from the list:

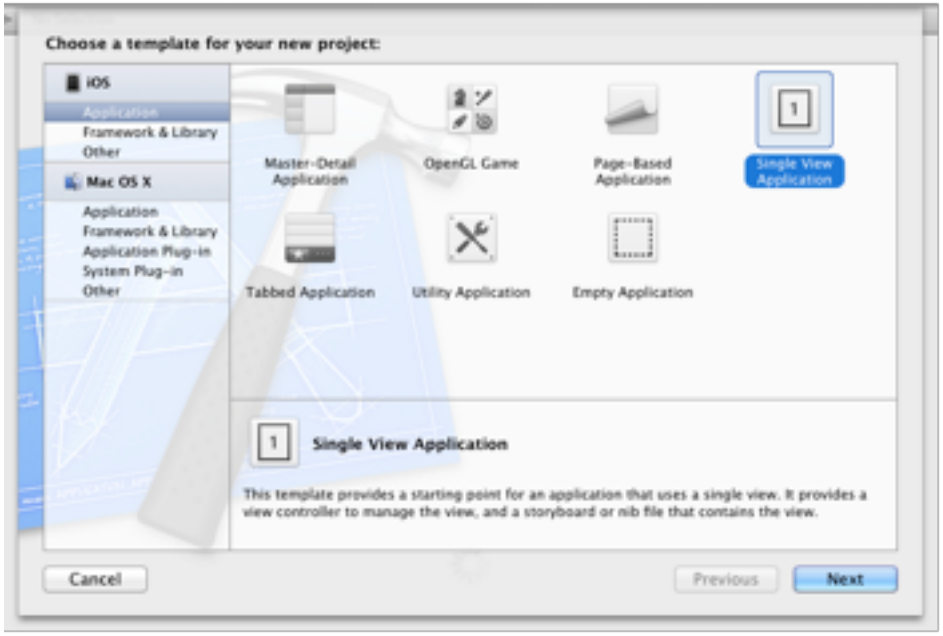


Figure 3 – Select the Single View Application template

Give your new application a name, and make sure that the *Use Storyboard* option is selected:

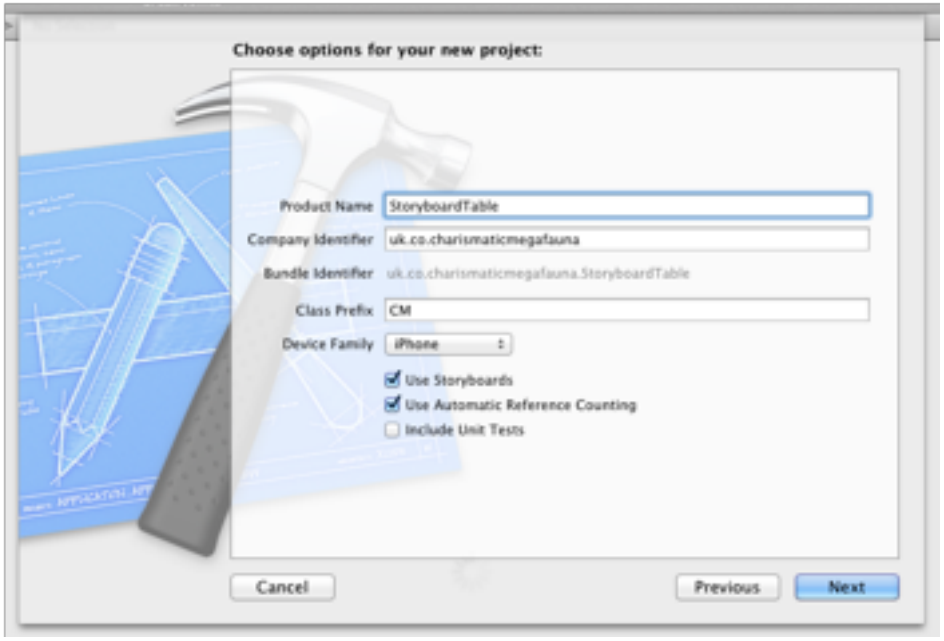


Figure 4 – Setting up the application's options

Then click **Next** and choose a location for the project on disk.

An template application will be created with an app delegate, a view controller and a storyboard:

Figure 5 – The template application

The storyboard that's created for you contains a blank view controller scene:

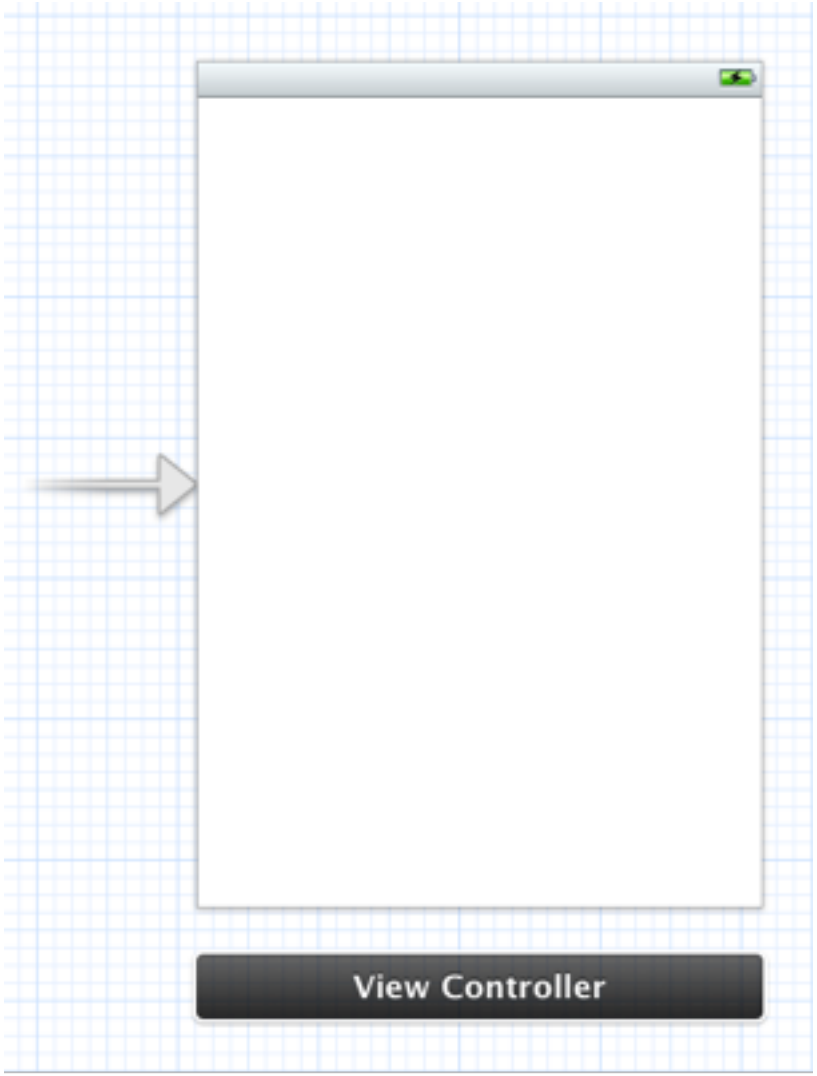


Figure 6 – The default empty scene

Adding the tab bar controller

The default view controller that the template provides is no use to us, so delete it by clicking in its status bar, or clicking the View Controller object in the object list at the left-hand side (if you click the scene itself, make sure that the whole thing has a blue highlight around it before you hit delete).

Then from the Objects list on the right-hand side, drag a Tab Bar Controller into the main window – this will place three scenes: the tab bar controller itself, and two empty view controllers (one for each of the two tabs in the default tab bar controller):

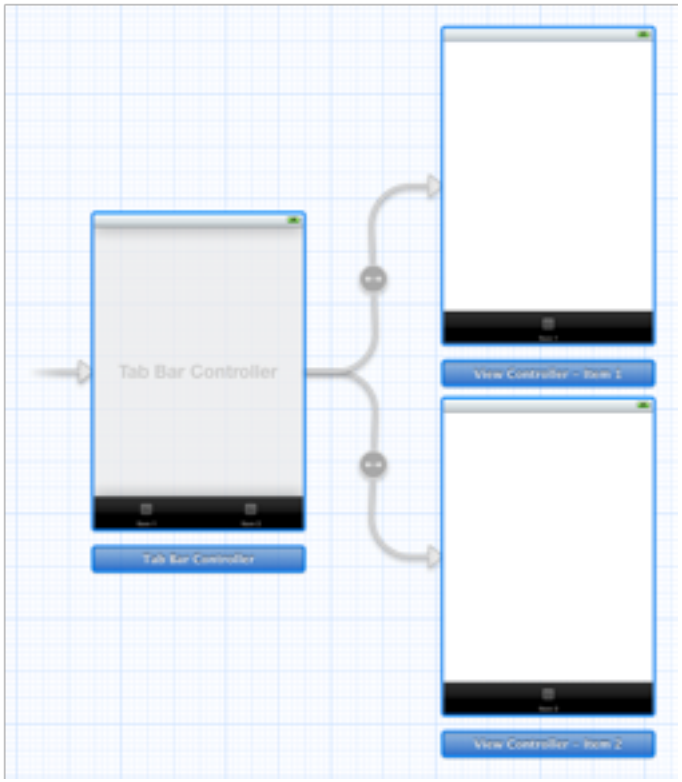


Figure 7 – The default tab bar controller

Drag a few controls onto each of the two view controllers, then run the app in the Simulator. You'll see that you get two tabs, and can use these to switch between the two view controllers:

Figure 8 – The skeleton app

That's not a bad result for a bit of dragging and dropping – especially compared with the amount of work involved in creating a tab bar application the "traditional" way – but it's not quite what we're after.

Adding the navigation controller and table view

We need to replace one of the view controllers with a navigation controller, inside which will sit a table view. To achieve this, first select and delete one of the view controllers (it doesn't matter which one, but I've got rid of the first.

Then drag a Navigation Controller object from the list at the right into the main window – this brings two scenes with it, the navigation controller itself, and a table view controller:

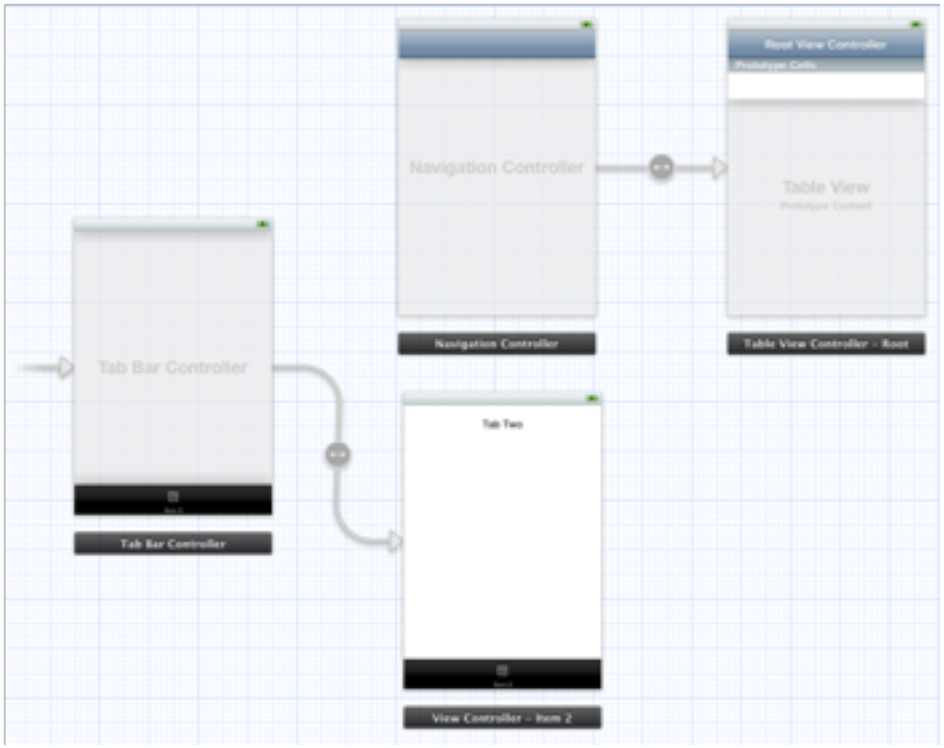


Figure 9 – The Navigation Controller object in place

As it stands, this new navigation controller isn't connected to anything – so if you run the app at this point you'll only see a single tab. To connect it up, to the Tab Bar Controller, ctrl-click in the Tab Bar Controller scene and drag the connection line out to the Navigation Controller. When you release the mouse button, the Storyboard Seques HUD will pop up:

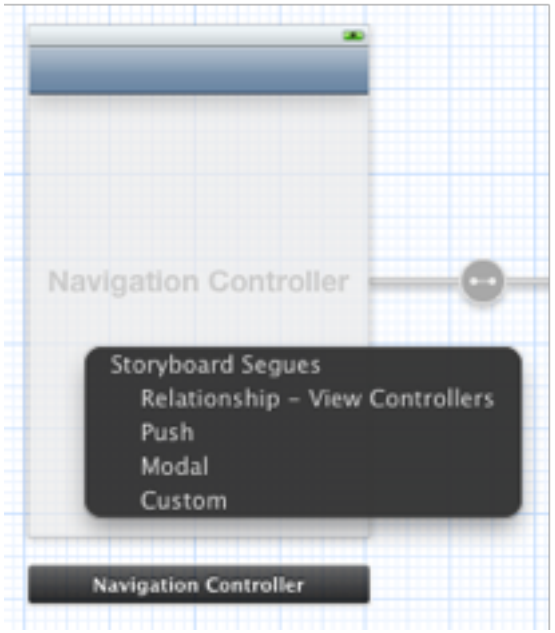


Figure 10 – The Storyboard Seque HUD

Select the Relationship-View Controllers option from the list, and the tab bar controller will be linked to the navigation controller, as shown in Figure 11:

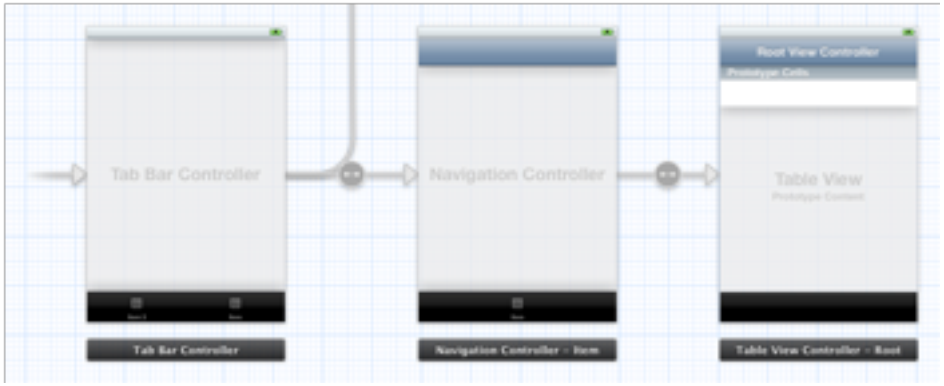


Figure 11 – The two controllers linked together

Because we deleted the first view controller, the new navigation controller will be placed in the second tab – if you want to change that, just reorder the tabs by dragging them around in the Tab Bar Controller's scene.

If you run the app now, you'll see that the first tab now displays a tableView (that's shown in Figure 12):

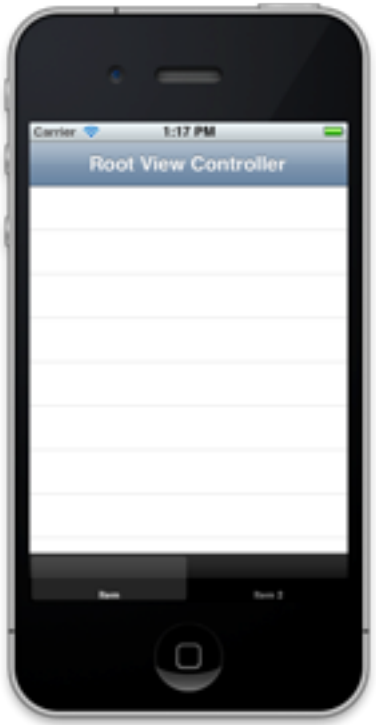


Figure 12 – The table view in the first tab

Showing data in the table view

Impressive as this is, we're not quite there yet – there's no data in the table yet! In pre-Storyboard terms, we've dropped a `UITableView` object into a NIB file – so the `tableView` is yet to have a datasource or delegate, so hasn't got anything to display. Let's fix that.

We need an object or objects to act as datasource and delegate – we could roll one from scratch, but it'll be much quicker to create a new instance of a `UITableViewController` and connect the `tableView` to this.

Create a new file (**File > New > File**) or **Cmd-N** and select the *Objective-C Class* option from the list of templates. Then use the dropdown list in the next dialogue to change the subclass to `UITableViewController`. Give the file a name (I've called mine `CMTableViewController`) and click **Next**, then **Create**.

> **Note** - You don't need a XIB file, so leave that option unchecked.

This will create a stubbed-out `UITableViewController` subclass with all the main methods implemented in boilerplate. To get this up and running, let's amend the key methods to create some cells with dummy data.

Listing 1 – the numberOfSectionsInTableView: method

```
- (NSInteger)numberOfSectionsInTableView:(UITableView *)ta-
bleView
{
    // Return the number of sections.
    return 1;
}
```

Listing 2 – the tableView:numberOfRowsInSection: method

```
- (NSInteger)tableView:(UITableView *)tableView
numberOfRowsInSection:(NSInteger)section
{
    // Return the number of rows in the section.
```

```
    return 20;
}
```

Listing 1 tells the `tableView` that it has a single section, and listing 2 tells it that there will be 20 rows.

Listing 3 is the point where the cells are created and handed back to the `tableView` itself:

Listing 3 – the initial tableView:cellForRowAtIndexPath: method

```
- (UITableViewCell *)tableView:(UITableView *)tableView
cellForRowAtIndexPath:(NSIndexPath *)indexPath
{
    static NSString *CellIdentifier = @"Cell";
    UITableViewCell *cell = [tableView
dequeueReusableCellWithIdentifier:CellIdentifier];

    // Configure the cell...

    return cell;
}
```

As it stands, this really doesn't do a great deal – so let's tweak it so that it creates some row-specific dummy content in each cell, shown in Listing 4:

Listing 4 – the revised tableView:cellForRowAtIndexPath: method

```
- (UITableViewCell *)tableView:(UITableView *)tableView
cellForRowAtIndexPath:(NSIndexPath *)indexPath
{
    static NSString *CellIdentifier = @"standardCell";
    UITableViewCell *cell = [tableView
dequeueReusableCellWithIdentifier:CellIdentifier];

    // Configure the cell...

    cell.textLabel.text = [NSString stringWithFormat:@"This
is row %d", indexPath.row];

    return cell;
}
```

This simple creates (or dequeues) a cell with an identifier of `standardCell`, and returns it having set the `textLabel` to show the row number.

If you run the app now, you won't see any change – the `tableView` will still be full of blank cells. You need to connect the `tableView` to the `UITableViewController` subclass that you just created so that it can act as the `delegate` and `dataSource`.

Switch back to the Storyboard view, and highlight the table view controller so that it's surrounded by the blue highlight (the Table View Controller object will be selected in the left-hand objects list, as shown in Figure 13):

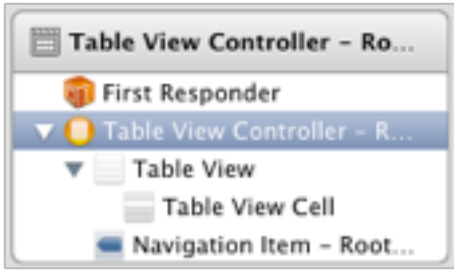


Figure 13 – the Table View Controller selected

Then switch your attention over to the *Identity Inspector* over at the right hand side, and drop down the list in the *Custom Class* section:

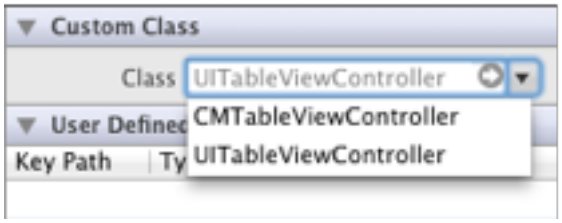


Figure 14 – the Custom Class list

That will show the new `UITableViewController` subclass that you've just created – select that, and your new class will now act as a `dataSource` and `delegate` for the `tableView`.

There's one last thing to do – if you run the app now, it'll crash with an error looking something like this:

```
StoryboardTable[2449:f803] *** Terminating app due to un-
caught exception 'NSInternalInconsistencyException', rea-
son: 'UITableView dataSource must return a cell from
tableView:cellForRowAtIndexPath:'
```

This somewhat-cryptic error is because we haven't yet told the `tableView` what cellIdentifiers it should be asking for. We altered the cellIdentifier value in the `tableView:cellForRowAtIndexPath:` method to label the cells with an identifier of `standardCell`, but the `tableView` doesn't know this yet.

That's fixed by highlighting the *Table View Cell* item in the list (see Figure 15):

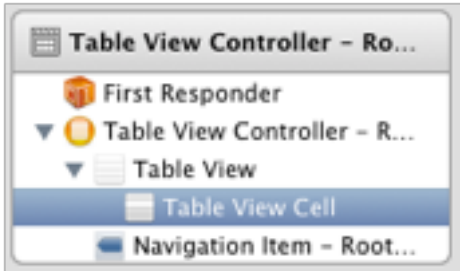


Figure 15 – Highlighting the Table View Cell

and altering the *Identifier* value to `standardCell` in the *Attributes Inspector* (see Figure 16):

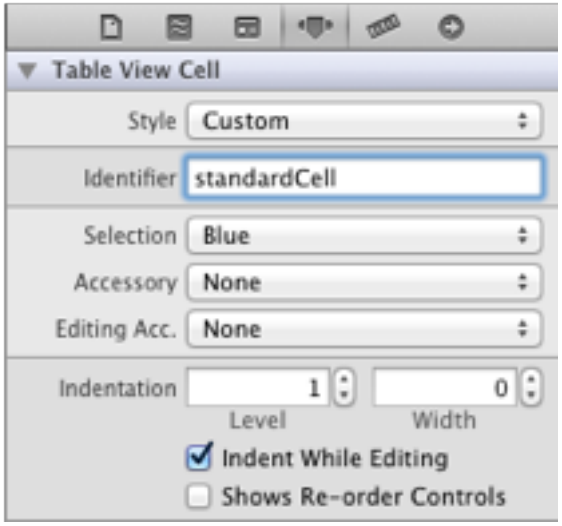


Figure 16 – The cell identifier

Save this and re-run the app, and now you'll see our `tableView` replete with real, live cells (see figure 17):

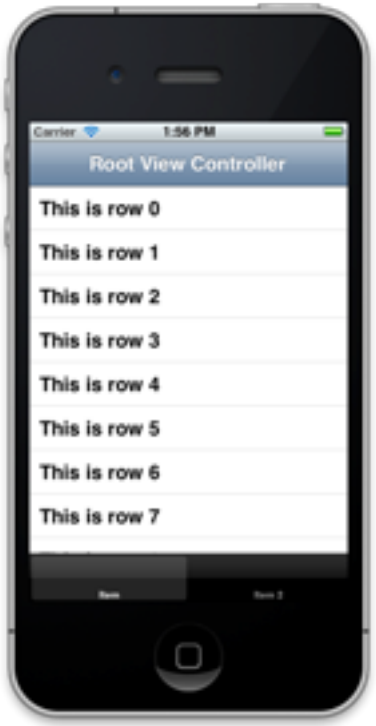


Figure 17 – The app, with live cells

A last little bit of housekeeping – to change the title in the navigation bar, highlight the *Navigation Item* in the list (see Figure 18):

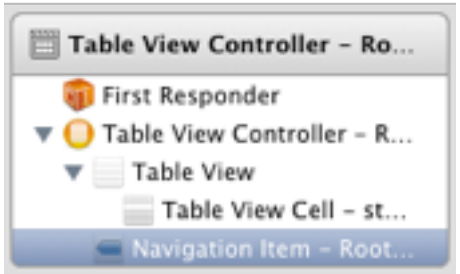


Figure 18 – Selecting the Navigation Item

And then change the *Title* value in the *Attributes Inspector* (shown in Figure 19):

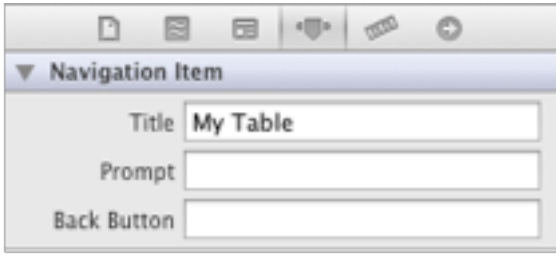


Figure 19 – The title

Summary

In this section, we've covered the process of creating a navigation-based app using Storyboards. First, we've looked at creating a Storyboard-based app. Next, we saw how to add and connect views with tab bar controllers and segues. Finally, we build a table-based navigation interface by wiring up table view controllers with the appropriate segues.

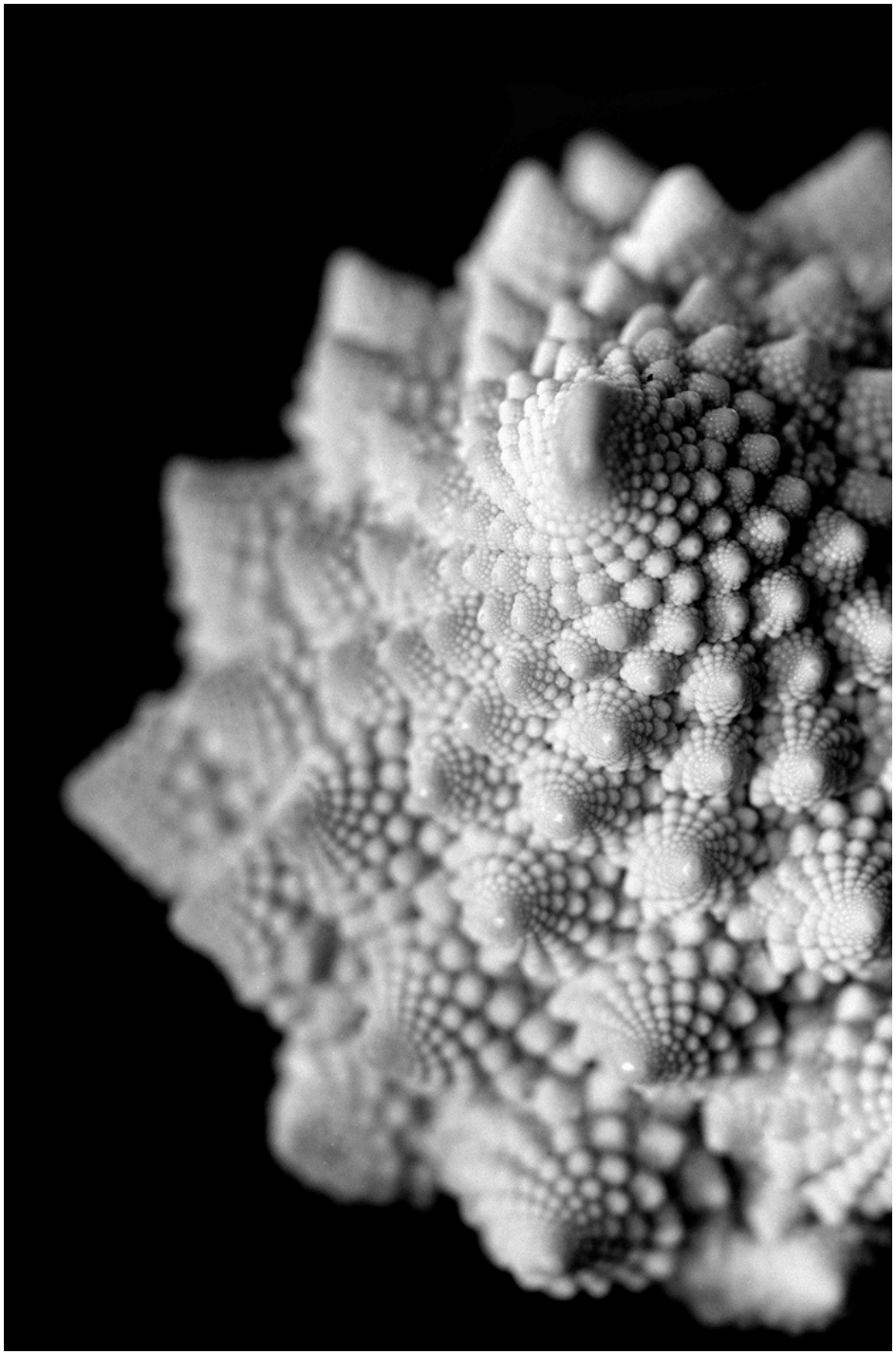
From here, you can extend things by building up detail views, or extending the table-based navigation to drill down further into the data.

Chapter 3

Customizing Cells With Storyboards

Although the standard iOS table views provide a range of built-in cell layouts, it doesn't take long before the need to customise these arises.

Storyboards provide prototype cells, that eases the process of creating custom cell styles. In this section, we'll look at the process involved with this.



Customising Cells with Storyboards

When you place a Table View into a Storyboard, it comes with a prototype cell – an empty cell that can either be used as-is with one of four standard `UITableViewCell` types; or as the basis of your own customisations.

Figure 20 – The standard cell styles

Figure 20 shows the standard cell styles:

- Basic
- Right Detail
- Left Detail
- Subtitle

The text in the cell can be set by setting the `textLabel` and `detailTextLabel` properties.

Although cell prototypes are a really useful feature of iOS5, one limitation that you need to bear in mind is that you're limited to one prototype cell per table. If you need more than one cell style in the same table – a different one for odd and even rows, perhaps – then you'll need to stick with the established NIB and subclass process.

Customizing prototype cells

If the four standard cell types aren't to your taste, you can customize the cell completely by setting the style to *Custom*, and placing your own controls.

Because any custom controls you place in the cell won't be known to the `UITableViewController`, you need some way for the controller to be able to "reach into" the cell and set the custom control properties.

There are two approaches to do this:

1. Giving each custom control a unique tag, and using the tag as the reference to the control in the `UITableView-Controller`

2. Creating a custom subclass of `UITableViewCell` with outlets for the custom controls, and creating instances of this custom cell to populate the table

Either approach is equally valid, but I tend to go for the second one as it provides an extra degree of flexibility and results in somewhat "cleaner" code.

Custom controls and tagging

Using tagging with custom controls is pretty straight forward. There are three stages:

1. Lay out your custom controls in the prototype cell

2. Give each custom control a unique tag

3. When creating the cell in the `tableView:cellForRowAtIndexPath:` method, use the tag property to get a reference to each custom control, and set the control's properties as required.

I'll not go into details about the first step, as this will obviously depend on the requirements of your app's UI. When you've placed the custom control, you set its `tag` property in the *Tag* box in the *View* section of the Attributes Inspector (see Figure 21):

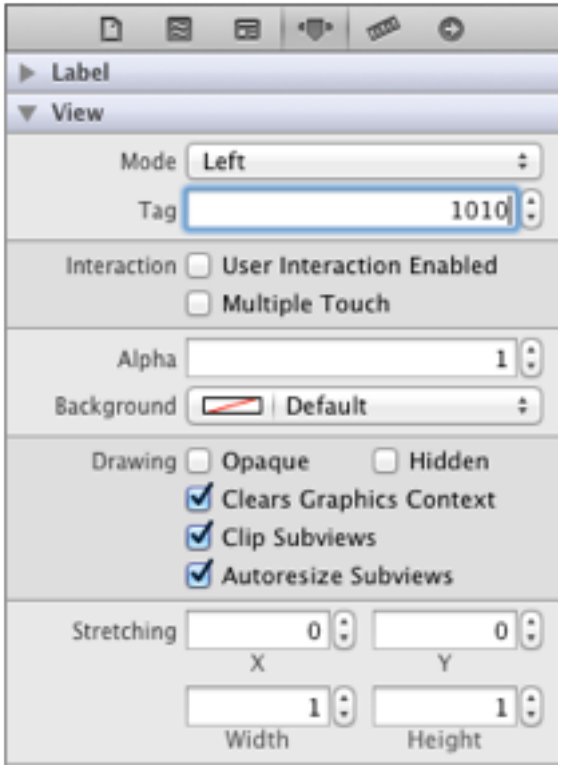


Figure 21 – Setting the custom control's Tag value

Each custom control's `tag` value must be unique, or very weird things will start to happen when you attempt to set the control's properties.

To access the control via its tag value, you need to do a little bit of `UIView` archeology as you create the cell – Listing 5 shows an example of this:

Listing 5 – Setting custom control properties using tag values

```
- (UITableViewCell *)tableView:(UITableView *)tableView
cellForRowAtIndexPath:(NSIndexPath *)indexPath
{
    static NSString *CellIdentifier = @"customCell";
```

```
    UITableViewCell *cell = [tableView
dequeueReusableCellWithIdentifier:CellIdentifier];

    // Configure the cell...
    UILabel *titleLabel = (UILabel *)[cell
viewWithTag:1010];
    titleLabel.text = [NSString stringWithFormat:@"This is
row %d", indexPath.row];

    UILabel *subTitleLabel = (UILabel *)[cell
viewWithTag:1020];
    subTitleLabel.text = [NSString
stringWithFormat:@"Subtitle for row %d", indexPath.row];

    UIImageView *cellIcon = (UIImageView *)[cell
viewWithTag:1030];
    [cellIcon setImage:[UIImage
imageNamed:@"theIcon.png"]];

    return cell;
}
```

Note `UITableViewCell`'s `viewWithTag:` method returns a `UIView`, so you'll need to cast this into the appropriate object type before you can set its properties.

Subclassing UITableViewCell

Using tags to access custom controls is a valid approach, but relying on tag values has always felt somewhat fragile to me. The alternative approach is to create a custom subclass of `UITableViewCell`, and connect the custom controls in your cell up to outlets in the subclass.

The steps involved in this approach are:

1. Lay out the controls in the cell as before
2. Create a subclass of `UITableViewCell`, and create outlets for each control in the cell that you want to configure at runtime
3. Make the prototype cell in the table view scene an instance of your custom cell
4. Connect the controls to the outlets
5. Configure the subclass's outlet properties in the `tableView:cellForRowAtIndexPath:` method

On the face of it, this sounds quite involved – but it's actually pretty straight forward in practice.

Laying out the controls needs no introduction, so let's go straight to creating the subclass of `UITableViewCell`. Create a new file (**File > New > File**), select *Objective-C Class* from the list of templates, give the subclass a name, and select `UITableViewCell` as its parent (see Figure 22):

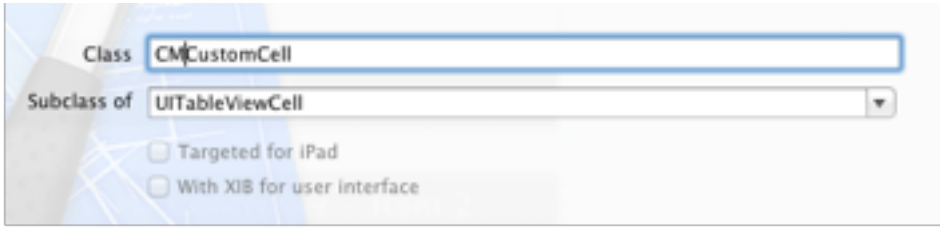


Figure 22 – Creating the custom subclass

With the subclass created, you can add outlets to it – here's an example in Listing 6:

Listing 6 – An example `CMCustomCell.h`

```
#import <UIKit/UIKit.h>
@interface CMCustomCell : UITableViewCell

@property (nonatomic, strong) IBOutlet UILabel
   *cellTitleLabel;
@property (nonatomic, strong) IBOutlet UILabel
   *cellSubtitleLabel;
@property (nonatomic, strong) IBOutlet UIImageView
   *cellIcon;

@end
```

You'll need to synthesize these outlets in the normal way in `CMCustomCell.m`.

Having created the outlets, you're now ready to hook them up to the prototype cell. The first step is to set the prototype cell's class to your custom subclass – in the *Custom Class* section of the Attributes Inspector, select your custom subclass from the dropdown list (it will automatically appear in this list because you created it as a subclass of `UITableViewCell`).

Now you can connect the controls to the outlets. In the Table View Controller's objects list on the left (see figure 23), ctrl-click the *Custom Cell* object and drag across to the first control.

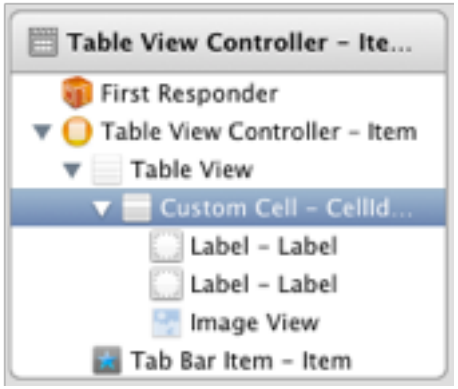


Figure 23 – The Table View Controller's object list

You'll be rewarded with the Outlets HUD, at the bottom of which will be all your custom subclass's outlets (as shown in Figure 24):

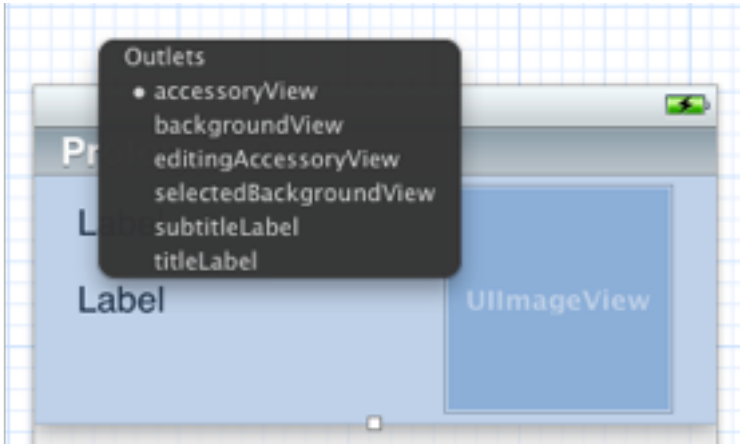


Figure 24 – The Outlets HUD

Select the appropriate outlet, and the control will be connected.

Once all your custom controls are connected, it's time to switch over to the `UITableViewController` class and amend the `tableView:cellForRowAtIndexPath:` method.

Firstly, add the `UITableViewCell` subclass as an import at the top of the `UITableViewController's` implementation file:

```
#import "CMCustomCell.h"
```

Then amend the `tableView:cellForRowAtIndexPath:` method along the lines of Listing 7:

Listing 7 – The amended tableView:cellForRowAtIndexPath: method

```
- (UITableViewCell *)tableView:(UITableView *)tableView
cellForRowAtIndexPath:(NSIndexPath *)indexPath
{
    static NSString *CellIdentifier = @"CMCustomCell";

    CMCustomCell *cell = (CMCustomCell *)[tableView
dequeueReusableCellWithIdentifier:CellIdentifier];

    cell.cellTitleLabel.text = [NSString
stringWithFormat:@"This is row %d", indexPath.row];
    cell.cellSubtitleLabel.text = [NSString
stringWithFormat:@"Subtitle row %d", indexPath.row];
    cell.cellIcon.image = [UIImage
imageNamed:@"cellicon.png"];

    return cell;
}
```

Let's step through this. The first line creates a `CellIdentifier` string which is used to identify what type of cell is returned from the cache. Next, the `tableView`'s `dequeueReusableCellWithIdentifier:` method attempts to return a cached cell, and creates a new one if the cache is empty. The method returns it as a standard `UITableViewCell` instance, so we need to cast that to an instance of `CMCustomCell`.

Having got a cell, we can then configure its outlet properties. There are three here – `cellTitleLabel`, `cellSubtitleLabel` and `cellIcon` – of course, yours will depend on the specific needs of your app.

Once configured, the cell can be returned by the method to be displayed in the table.

There's one last step to cover before the app will run. Flip back to the prototype cell and change its `Identifer` so that this matches the `CellIdentifier` string you set up in the `tableView:cellForRowAtIndexPath:` method (see Figure 25):

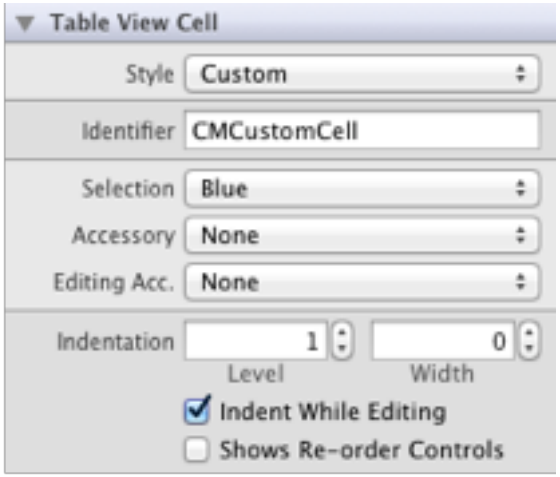


Figure 25 – Setting the cell identifier

If you run the app now, you'll see your table displaying the custom cells something like Figure 26, with properties configured appropriately:

Figure 26 – The completed app with custom cells

Summary

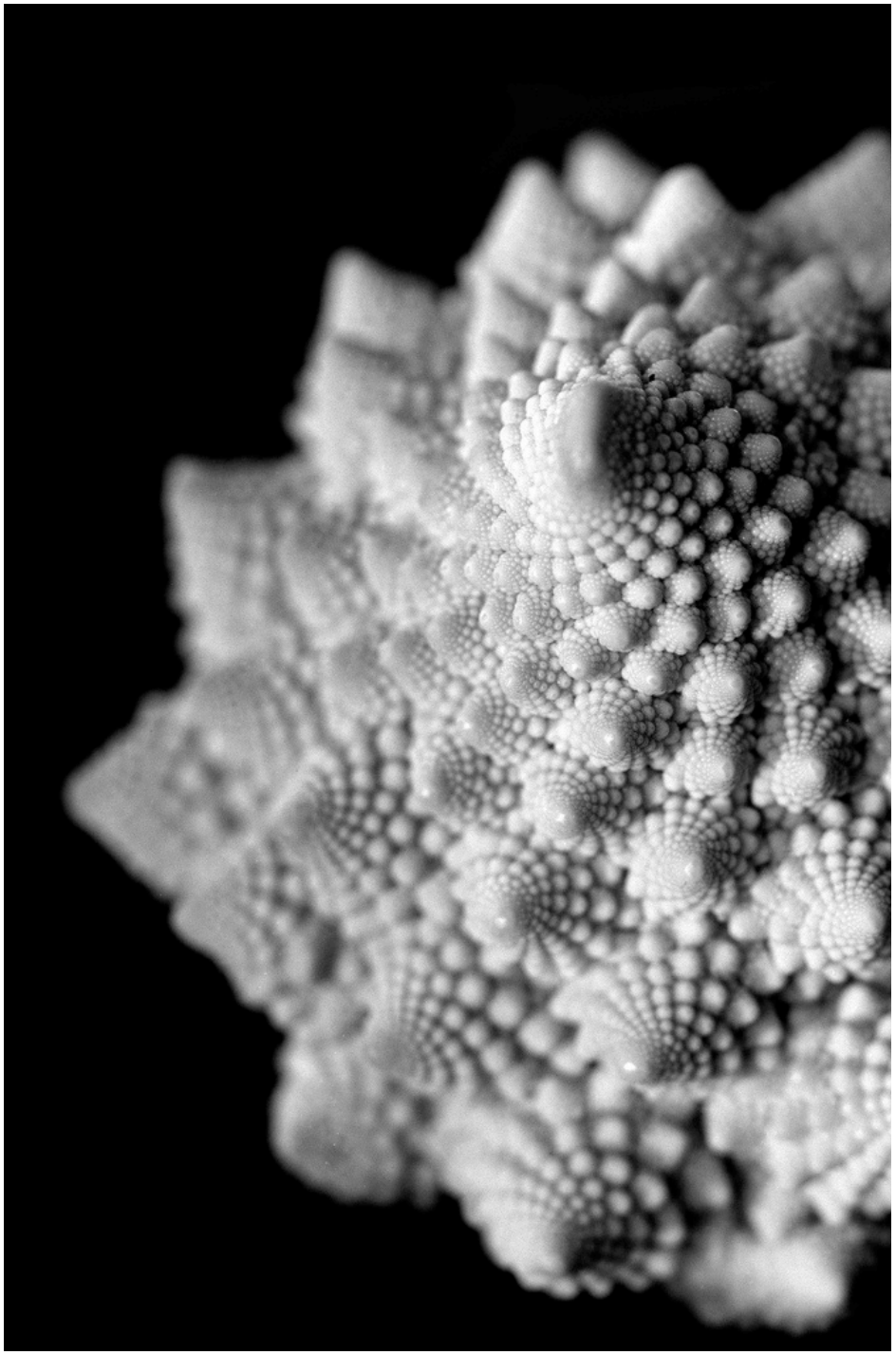
iOS5 and Xcode 4.2 introduced Storyboards, a layout tool that aids the rapid creation of user interface views and eases the complexity of building internal app navigation flows.

Storyboards provide a quick way of creating multi-view applications and wiring the different views together. In the case of table views, they also simplified the process of embedding them in navigation controllers.

The storyboard-based table view controller provides a prototype cell, which speeds up customization. You can use one of the four default cell styles; or add custom controls to the prototype cell. Once created, the custom cells controls can be referenced either by exploiting their `UIView` tag properties; or by using them as outlets for a custom `UITableViewCell` subclass.

Chapter 4

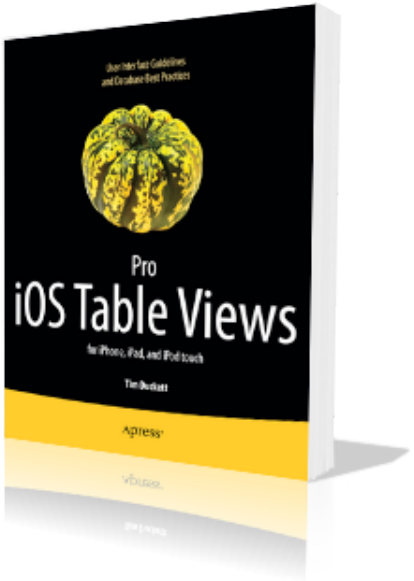
Further information



Further information

More information and detailed examples on how table views can be used in iOS applications can be found in my book:

Pro iOS Table Views for iPhone, iPad and iPod Touch



Published by Apress, Inc
ISBN 978-1-4302-3348-0

Further details can also be found at my site:

http://adoptioncurve.net